MODULE 02

Functions, Objects & Interfaces

The most important module for day-to-day TypeScript. You type functions and objects in nearly every file you write.

~50 min read | Builds on Module 1 | Core daily TS skills

1. Typing Function Parameters & Return Types

In JavaScript you write functions and hope the caller passes the right thing. In TypeScript, you declare exactly what goes in and what comes out. The compiler then enforces it everywhere the function is called.

JavaScript	TypeScript
`function add(a, b) {`	`function add(a: number, b: number): number {`
`  return a + b;`	`  return a + b;`
`}`	`}`

The pattern is simple: type each parameter, then add : ReturnType after the closing parenthesis.

Basic typed function

```
// Anatomy of a typed function
//                  param types        return type
//                  |          |       |
function greet(name: string, age: number): string {
  return `Hello ${name}, you are ${age} years old`;
}

greet('Alice', 30);       // OK
greet('Alice', '30');     // Error: '30' is not a number
greet(30, 'Alice');       // Error: arguments are in wrong order
```

> **Why type the return value?**
>
> If you don't specify a return type, TS infers it. But explicitly typing it is a safety net — TS will error if you accidentally return the wrong thing inside the function. It also makes the function's contract crystal clear to anyone reading it.

2. void and never — Two Special Return Types

void — Functions That Return Nothing

When a function doesn't return anything (like an event handler or a logger), its return type is void:

void return type

```
function logMessage(msg: string): void {
  console.log(msg); // no return statement
}

// You've used void-like functions your whole JS career:
// addEventListener callbacks, setTimeout callbacks, forEach callbacks
document.addEventListener('click', (e): void => {
  console.log(e.target);
});
```

never — Functions That Never Return

never is for functions that literally never finish — they always throw an error or run an infinite loop. This is rare but important:

never return type

```
function crashApp(msg: string): never {
  throw new Error(msg); // always throws, never returns
}

function infiniteLoop(): never {
  while (true) { /* runs forever */ }
}
```

3. Optional Parameters & Default Values

Just like in JS, you can have optional parameters. In TS, mark them with ? after the name. Default values work exactly the same as JS — and TS automatically infers the type from the default:

Optional & Default parameters

```
// Optional parameter — must come AFTER required params
function createUser(name: string, role?: string): string {
  return `${name} is a ${role ?? 'guest'}`;
}

createUser('Alice');           // OK — role is undefined
createUser('Alice', 'admin');  // OK

// Default parameter — TS infers type from the default value
function createUser2(name: string, role = 'guest'): string {
  // role is inferred as string (from 'guest')
  return `${name} is a ${role}`;
}

createUser2('Bob');            // role = 'guest'
createUser2('Bob', 'admin');   // role = 'admin'
createUser2('Bob', 42);        // Error: number not assignable to string
```

4. Typing Objects Inline

In JS, objects are just... objects. In TS, you describe their shape — what properties they have and what types those properties are:

Inline object typing

```
// JS: any shape accepted, no enforcement
function printUser(user) {
  console.log(user.name, user.age);
}

// TS: describe the shape inline with { }
function printUser(user: { name: string; age: number }): void {
  console.log(user.name, user.age);
}

printUser({ name: 'Alice', age: 30 });        // OK
printUser({ name: 'Alice' });                  // Error: age is missing
```

```
printUser({ name: 'Alice', age: 30, x: 1 });  // Error: 'x' not in type
```

> **Inline types get messy fast**
>
> Typing objects inline works, but if you use the same shape in multiple places, it becomes a copy-paste nightmare. That's exactly what interfaces and type aliases solve — which we cover next.

5. Interfaces — Naming Your Object Shapes

An interface is a named description of an object's shape. Think of it as a contract: any object that claims to be this type must have these properties.

Interface definition and usage

```
// Define the shape once
interface User {
  id:       number;
  name:     string;
  email:    string;
  age?:     number;   // optional property
}

// Use it everywhere
function greetUser(user: User): string {
  return `Hi ${user.name}!`;
}

function saveUser(user: User): void {
  // save to database...
}

const alice: User = { id: 1, name: 'Alice', email: 'a@a.com' };  // OK
const bob: User   = { id: 2, name: 'Bob' };  // Error: email missing
```

Readonly Properties

You can mark interface properties as readonly so they can't be changed after the object is created — great for IDs and immutable data:

Readonly properties

```
interface Product {
```

```
  readonly id:   number;  // can never be changed after creation
  name:          string;
  price:         number;
}

const laptop: Product = { id: 1, name: 'Laptop', price: 999 };
laptop.name = 'Gaming Laptop';  // OK — name is mutable
laptop.id   = 2;                // Error: id is readonly
```

6. interface vs type — When to Use Which

Both can describe object shapes. Here's the practical difference:

Feature	interface	type alias
Describe object shapes	Yes	Yes
Can be extended (inheritance)	Yes (extends keyword)	Yes (& intersection)
Can describe unions/primitives	No	Yes
Can be merged/re-opened	Yes (declaration merging)	No
Best for…	Objects, classes, React props	Unions, primitives, complex types

> **Practical rule of thumb**
>
> Use interface for objects and React component props (it gives better error messages). Use type for unions, primitives, and anything that isn't a plain object shape. When in doubt, interface is the safer default for objects.

7. Extending Interfaces — Inheritance

Interfaces can extend other interfaces, building more specific types from general ones. This is the same concept as class inheritance, but for shapes:

Extending interfaces

```
interface Animal {
  name:  string;
  age:   number;
}
```

```
interface Dog extends Animal {
  breed:   string;  // Dog has everything Animal has, PLUS this
  isGood:  boolean; // (always true)
}

const rex: Dog = {
  name:   'Rex',     // from Animal
  age:    3,         // from Animal
  breed:  'Husky',   // Dog-specific
  isGood: true,      // Dog-specific
};

// You can also extend multiple interfaces at once:
interface AdminUser extends User, Timestamps {
  permissions: string[];
}
```

8. Typing Arrow Functions

You use arrow functions constantly in React. Here's how to type them — the syntax is the same as regular functions:

Arrow function types

```
// Regular arrow function
const double = (n: number): number => n * 2;

// Arrow function stored in a variable — explicit type on the variable
const greet: (name: string) => string = (name) => `Hello ${name}`;

// Multi-line arrow function
const add = (a: number, b: number): number => {
  const result = a + b;
  return result;
};

// In callbacks (very common in React)
const numbers = [1, 2, 3];
const doubled = numbers.map((n: number): number => n * 2);
// TS infers n is number from the array — you can often skip it:
const doubled2 = numbers.map(n => n * 2); // also fine
```

9. Putting It All Together — A Real-World Example

Here's a realistic piece of code combining everything from this module. This is the kind of TypeScript you'll write every day as a React developer:

Real-world typed module

```
// A complete typed API layer

interface Address {
  street:  string;
  city:    string;
  country: string;
}

interface User {
  readonly id:  number;
  name:         string;
  email:        string;
  address?:     Address;   // optional nested object
  role:         'admin' | 'user' | 'guest';  // union type
}

// Function that returns a User or null (not found)
function findUser(id: number): User | null {
  // simulating a lookup...
  return null;
}

// Function to update a user's role
function updateRole(user: User, newRole: User['role']): void {
  //                              ^^ access type from interface
  console.log(`Updating ${user.name} to ${newRole}`);
}

const user = findUser(1);
if (user) {
  updateRole(user, 'admin');    // OK
  updateRole(user, 'superuser'); // Error: not in the union
}
```

Module Summary

Everything you learned in this module:

- Type function params: `function f(x: string, y: number)`
- Type return values: `: string` after the closing parenthesis
- Use `void` for functions that return nothing, `never` for functions that always throw
- Optional params: `role?: string` — must come after required params
- Inline object types describe shape, but get repetitive quickly
- interface names object shapes, can be extended, best for objects & props
- type alias is more flexible — handles unions, primitives, complex types
- Extend interfaces: `interface Dog extends Animal { }`
- Arrow functions use the same type syntax as regular functions

> Next: Module 3

Generics — Write Code Once, Use for Any Type

The single most powerful feature in TypeScript. Generic functions, generic interfaces, constraints, and real-world patterns you'll see in every React codebase.